

Sesiunea 2: Planificarea Testării (Test Plan)

Capitolul 1: Ce este un Plan de Testare și de ce este vital?

1. Definiția Planului de Testare (Test Plan)
 2. De ce avem nevoie de un Plan de Testare? (Justificarea investiției)
 3. Diferența dintre Strategia de Testare și Planul de Testare
- 💡 Exemple Practice pentru Discuție

Știați că...?

Poveste Aplicată: „Planificarea unei Expediții pe Everest”

🧐 Activitate de reflexie: „Ce punem în rucsac?”

Capitolul 2: Structura Standard a unui Plan de Testare (Analiză pe Secțiuni)

1. Obiectivul și Domeniul de Aplicare (Scope)
2. Roluri și Responsabilități (Roles & Responsibilities)
3. Programul și Jaloanele (Schedule & Milestones)
4. Criteriile de Intrare și ieșire (Entry & Exit Criteria)
5. Criterii de Suspendare și Reluare

💡 Exemple Practice pentru Discuție

Știați că...?

Poveste Aplicată: „Organizarea unui Eveniment de Nuntă”

🧐 Exercițiu de Gândire: „OrangeHRM în Acțiune”

Capitolul 3: Analiza Cerințelor de Business și Derivarea Condițiilor de Testare

1. Ce sunt Cerințele de Business (Requirements)?
2. De ce analizăm cerințele exhaustiv?
3. Derivarea Condițiilor de Testare (Test Conditions)

💡 Exemple Practice pentru Discuție (OrangeHRM)

Știați că...?

Poveste Aplicată: „Rețeta Bucătarului și Clientul Pretențios”

🧐 Exercițiu de Gândire: „Detectivul de Cerințe”

Capitolul 4: Test Case Design (Anatomia și Scrierea Cazurilor de Test)

1. Ce este un Caz de Test?

2. Structura Exhaustivă a unui Caz de Test (Anatomia)

- A. Test Case ID (Identificatorul Unic)
- B. Titlu / Sumar (Title / Summary)
- C. Precondiții (Preconditions)
- D. Pașii de Testare (Test Steps)
- E. Date de Test (Test Data)
- F. Rezultat Așteptat (Expected Result) - Cel mai important câmp!
- G. Rezultat Real (Actual Result)
- H. Post-condiții (Post-conditions)

3. Proprietățile unui Test Case de Calitate (Standarde QualiAdept)

💡 Exemple Practice pentru Discuție (OrangeHRM)

Știați că...?

Poveste Aplicată: „Manualul de Asamblare IKEA”

🧑‍🔬 Exercițiu de Gândire: „Să scriem corect!”

Capitolul 5: Tipuri de Teste și Piramida Testării

1. Nivelurile de Testare (Testing Levels)

- A. Testarea Unitară (Unit Testing)
- B. Testarea de Integrare (Integration Testing)
- C. Testarea de Sistem (System Testing)
- D. Testarea de Acceptanță (Acceptance Testing - UAT)

2. Piramida Testării (The Testing Pyramid)

3. Retesting vs. Regression Testing (Marea Distincție)

💡 Exemple Practice pentru Discuție (OrangeHRM)

Știați că...?

Poveste Aplicată: „Construirea unui Smartphone”

🧑‍🔬 Exercițiu de Gândire: „Detectivul de Regresie”

Capitol Bonus: Testarea Exploratorie vs. Testarea Scriptată (Intuiție vs. Structură)

1. Testarea Scriptată (Scripted Testing)

2. Testarea Exploratorie (Exploratory Testing)

🧑‍🔬 Comparație: Când folosim ce?

💡 Exemple Practice pentru Discuție (OrangeHRM)

Știați că...?

Poveste Aplicată: „Turistul vs. Exploratorul”

🧑‍🔬 Exercițiu de Gândire: „Dincolo de pași”

Capitolul 6: Recapitulare, Concluzii și Proiectul OrangeHRM

1. Recapitulare Exhaustivă: Ce am învățat în Sesiunea 2?
 2. Concluziile QualiAdept: „Legile Strategului QA”
 3. Temă de Casă: „Proiectul OrangeHRM - Faza de Design” 🚀
 - Task 1: Fragment de Plan de Testare (Modulul PIM - Personal Information Management)
 - Task 2: Analiza de Cerințe (Cazul „Adăugare Angajat”)
 - Task 3: Design de Cazuri de Test (Execuție)
 - Task 4: Strategie de Regresie
- 💡 Sfat de final de la echipa QualiAdept

Capitolul 1: Ce este un Plan de Testare și de ce este vital?

🧠 În acest capitol, explorăm documentul central care guvernează întreaga activitate de QA într-un proiect. Dacă în Sesiunea 1 am învățat că suntem detectivi, în Sesiunea 2 învățăm să fim „șefi de operațiuni”. Un Plan de Testare nu este doar o formalitate birocratică, ci este harta care ne împiedică să ne pierdem în complexitatea software-ului și să depășim bugetele sau termenele limită.

1. Definiția Planului de Testare (Test Plan)

Definiție: Planul de Testare este un document detaliat care descrie strategia, obiectivele, programul, resursele (umane și tehnice) și scopul activităților de testare.

- **Rolul documentului:** Servește ca un „contract” între echipa de testare și restul organizației (Dezvoltatori, Manageri, Clienți).
- **Cine îl creează?** De obicei, Test Lead-ul sau un QA Senior, dar întreaga echipă contribuie la detaliile tehnice.
- **Când se creează?** În faza de planificare a proiectului, imediat după ce cerințele de business au fost clarificate.

2. De ce avem nevoie de un Plan de Testare? (Justificarea

investiției)

Fără un plan, testarea devine un proces subiectiv și inefficient. Iată motivele exhaustive pentru care nicio companie serioasă nu începe testarea fără un Plan:

- **Clarificarea Scopului (Scope):** Definește exact **ce testăm** și, la fel de important, **ce NU testăm**. (Ex: Testăm versiunea web, dar nu și cea de mobil).
- **Identificarea Resurselor:** Știm de câți oameni avem nevoie, ce telefoane/laptopuri trebuie să cumpărăm și ce acces la baze de date ne trebuie.
- **Evaluarea Riscurilor:** Anticipăm problemele (ex: „Dacă mediul de testare cade, cum recuperăm timpul pierdut?“).
- **Controlul Timpului:** Stabilește jaloane (Milestones) clare. Știm exact în ce zi începem și în ce zi trebuie să dăm verdictul final.
- **Trasabilitate și Transparență:** Oricine din proiect poate deschide Planul și poate vedea exact stadiul calității.

3. Diferența dintre Strategia de Testare și Planul de Testare

Aceasta este o întrebare capcană clasică la interviuri.

- **Strategia de Testare (Test Strategy):** Este un document de nivel înalt, de obicei la nivel de companie, care spune „cum testăm în general” (ex: „toate proiectele noastre folosesc Jira și necesită 80% acoperire”). Este statică.
- **Planul de Testare (Test Plan):** Este specific unui proiect. Spune „cum testăm aplicația OrangeHRM în sprintul de luna aceasta”. Este dinamic și se actualizează pe măsură ce proiectul evoluează.

💡 Exemple Practice pentru Discuție

Scenariul A: Proiectul fără „Harta” (Fără Plan)

- **Context:** O echipă de 5 QA primește o aplicație nouă. Managerul spune: „Doar găsiți bug-uri, nu pierdeți timpul cu documente”.
- **Consecința:** După 2 săptămâni, 3 testeri au verificat aceeași pagină de Login, nimeni nu a verificat procesarea plăților, iar mediul de testare a fost șters de un programator pentru că nu știa că QA-ul are nevoie de el.
- **Analiză QA:** Planul de testare ar fi alocat sarcini diferite fiecărui tester și ar fi „rezervat” mediul de testare, evitând risipa de timp.

Scenariul B: „Ce NU testăm” este salvator

- **Context:** În Planul de Testare este scris clar: „Testarea de securitate (Penetration Testing) nu face parte din scopul acestui proiect”.
- **Consecința:** La finalul proiectului, un client întreabă de ce nu s-au făcut teste de hacking.
- **Analiză QA:** Planul de Testare protejează echipa de QA. Dacă este semnat de client, acesta confirmă că a fost de acord cu limitările procesului.

Știați că...?

💡 Un Plan de Testare este considerat un **Document Viu**? Dacă pe parcursul proiectului se adaugă o funcționalitate nouă care nu era în planul inițial, documentul trebuie actualizat. Dacă ignorăm actualizarea planului, acesta devine o „bucată de hârtie inutilă” care nu mai reflectă realitatea din teren.

Poveste Aplicată: „Planificarea unei Expediții pe Everest”

Imaginează-ți că vrei să urci pe Everest.

- **Abordarea haotică:** Îți iei rucsacul și pleci. S-ar putea să uiți oxigenul, s-ar putea să nu ai destui oameni care să țină corturile sau s-ar putea să te prindă furtuna fără un adăpost planificat. Șansele de eșec sunt de 99%.
- **Abordarea cu Plan de Testare (Expediția Planificată):**
 - **Scop:** Ajungem pe vârf și ne întoarcem vii.
 - **Resurse:** 5 șerpași, 20 butelii de oxigen, mâncare pentru 30 de zile.
 - **Riscuri:** Avalanșe, degerături. Plan de rezervă: Dacă vremea e rea, așteptăm în Tabăra 2.
 - **Criterii de intrare:** Nu începem urcarea până nu primim prognoza meteo favorabilă.

Concluzia QualiAdept: Planul de Testare este acea planificare a expediției. El nu urcă muntele în locul tău (nu rulează testele), dar se asigură că atunci când ești pe munte, ai tot ce îți trebuie ca să nu eșuezi.

Activitate de reflexie: „Ce punem în rucsac?”

Gândește-te la aplicația **OrangeHRM**. Dacă ar trebui să planifici testarea modulului de „Concedii”

(Leave Management):

- Care ar fi cel mai mare **Risc** la care te poți gândi?
- Ce **Resurse** tehnice crezi că ți-ar trebui? (Ex: Acces la baza de date, acces la cont de Manager și cont de Angajat).

Capitolul 2: Structura Standard a unui Plan de

Testare (Analiză pe Secțiuni)



Un Plan de Testare profesionist urmează o structură logică, bazată adesea pe standardul internațional IEEE 829 (sau adaptări moderne ale acestuia). Fiecare secțiune are un rol critic în eliminarea ambiguității. Vom descompune acest document în cele mai importante „organe” ale sale, explicând exhaustiv ce trebuie să conțină fiecare.

1. Obiectivul și Domeniul de Aplicare (Scope)

Aceasta este „granița” proiectului tău. Fără un Scope bine definit, echipa de QA riscă să testeze prea mult sau prea puțin.

- **În Scop (In-Scope):** Lista funcționalităților care **vor fi** testate.
 - *Exemplu OrangeHRM:* „Modulul de Administrare Utilizatori, Modulul de Concedii (Leave Management), Logarea și Resetarea Parolei.”
- **În afara Scopului (Out-of-Scope):** Lista funcționalităților care **nu vor fi** testate. Este vital să scriem asta pentru a gestiona așteptările clientului.
 - *Exemplu OrangeHRM:* „Testarea de performanță la peste 10.000 de utilizatori simultani, Testarea pe browsere Internet Explorer (depășite), Modulul de Recrutare (care nu a fost încă livrat).”

2. Roluri și Responsabilități (Roles & Responsibilities)

Cine face ce? Într-o echipă QualiAdept, claritatea rolurilor previne situațiile de tipul „am crezut că face colegul”.

- **Test Manager/Lead:** Planifică, monitorizează riscurile, semnează documentul final.
- **QA Engineer (Tester):** Scrie cazurile de test, execută testele, raportează bug-urile.
- **Developer:** Repară bug-urile raportate și livrează versiuni noi (build-uri).
- **Product Owner (PO):** Clarifică cerințele de business și decide prioritățile.

3. Programul și Jaloanele (Schedule & Milestones)

Testarea nu poate dura la infinit. Trebuie să avem date calendaristice clare.

- **Milestone 1:** Finalizarea Planului de Testare (Data: X).
- **Milestone 2:** Finalizarea scrierii Test Case-urilor (Data: Y).

- **Milestone 3:** Finalizarea execuției (Data: Z).
- **Milestone 4:** Raportul final de testare (Data: W).

4. Criteriile de Intrare și ieșire (Entry & Exit Criteria)

Acestea sunt „vămile” procesului de testare. Ele ne spun când avem voie să începem și când avem voie să ne oprim.

- **Criterii de Intrare (Entry Criteria):** Ce trebuie să fie gata pentru a începe testarea?
 - *Exemplu:* Planul de testare aprobat, Mediul de testare funcțional, Codul livrat de developeri (fără erori de compilare).
- **Criterii de ieșire (Exit Criteria):** Când considerăm că am terminat?
 - *Exemplu:* Toate testele planificate au fost executate, 100% din bug-urile Critice și Majore sunt reparate și închise, Gradul de acoperire a cerințelor este de 100%.

5. Criterii de Suspendare și Reluare

Ce facem dacă „se rupe filmul”?

- **Suspendare:** Oprim testarea dacă apare un blocaj major.
 - *Exemplu:* Aplicația OrangeHRM se prăbușește (Crash) imediat după logare. Nu mai putem testa restul modulelor.
- **Reluare:** Când repornim?
 - *Exemplu:* După ce dezvoltatorii livrează un fix (reparare) documentat pentru problema care a cauzat suspendarea.

💡 Exemple Practice pentru Discuție

Scenariul A: „Misterul Bug-ului Neraportat”

- **Context:** Un tester găsește un bug de performanță (aplicația se mișcă lent). Managerul îl ceartă pentru că a pierdut 4 ore investigând asta.
- **Analiză QA:** Testerul verifică Planul de Testare la secțiunea **Scope**. Acolo scrie: „Testarea de performanță este Out-of-Scope”.
- **Concluzie:** Testerul a greșit irosind resurse pe ceva ce nu era planificat. Planul ne ajută să rămânem concentrați pe priorități.

Scenariul B: „Presiunea Lansării”

- **Context:** Clientul vrea să lanseze aplicația vineri. Avem 10 bug-uri deschise, din care 2 sunt

Critice.

- **Analiză QA:** Verificăm **Exit Criteria**. Acolo scrie clar: „Nu se poate lansa cu bug-uri Critice deschise”.
- **Concluzie:** Planul de Testare este argumentul tău legal în fața clientului. Tu nu spui „nu vreau eu să lansez”, ci spui „nu am îndeplinit criteriile de ieșire semnate de comun acord”.

Știați că...?

💡 În proiectele Agile, Planul de Testare este mult mai scurt și se concentrează adesea pe un singur Sprint (o perioadă de 2 săptămâni)? Nu mai scriem romane de 50 de pagini, ci documente sintetice numite **Test Summary** sau planuri de nivel „Lean”, pentru a ține pasul cu viteza de dezvoltare.

Poveste Aplicată: „Organizarea unui Eveniment de Nuntă”

Imaginează-ți că ești organizatorul unei nunți (QA Lead). Planul tău de testare este „Desfășurătorul Evenimentului”.

- **Scope (În Scop):** Ceremonia religioasă, Masa, Muzica. (Out-of-Scope: Transportul invitaților de la aeroport - de asta se ocupă altcineva).
- **Roles:** Bucătarii (Devs) gătesc, Ospătarii (Testers) gustă și verifică dacă mâncarea e caldă, Mirele (Product Owner) decide ordinea melodiilor.
- **Entry Criteria:** Nu începem masa până nu au ajuns toți invitații și mâncarea nu a fost livrată de furnizor.
- **Suspension Criteria:** Dacă se ia curentul (Blocaj), petrecerea se suspendă până vine echipa de mentenanță.
- **Exit Criteria:** Nunta se consideră încheiată când s-a tăiat tortul și invitații au primit cadourile de plecare.

Fără acest desfășurător, bucătarii ar putea găti felul doi înainte de aperitiv, sau muzica ar putea începe în timp ce mirele este încă la biserică. În QA, Planul de Testare asigură că „petrecerea” (lansarea software-ului) este un succes, nu un haos.

Exercițiu de Gândire: „OrangeHRM în Acțiune”

Imaginează-ți că trebuie să definești Criteriile de Ieșire pentru testarea modului de **Recrutare** din OrangeHRM:

- Ce procent de teste trebuie să fie „Passed”?
- Ce facem cu bug-urile de tip „Typo” (greșeli de scriere)? Le lăsăm să treacă sau blocăm lansarea?
- Care este condiția tehnică minimă ca un candidat să poată spune că procesul de testare e gata?

Capitolul 3: Analiza Cerințelor de Business și Derivarea Condițiilor de Testare

💰 Un tester de elită nu așteaptă să primească aplicația pentru a începe lucrul. El începe să

testeze încă de când primește documentația. Analiza cerințelor este o formă de **Testare Statică** unde „vânam” bug-urile logice înainte ca ele să fie programate. Dacă cerința este ambiguă, codul va fi greșit, iar testul tău va fi confuz. În acest capitol, învățăm cum să „disecăm” o cerință pentru a extrage **Condițiile de Testare**.

1. Ce sunt Cerințele de Business (Requirements)?

Cerințele reprezintă descrierea a ceea ce trebuie să facă sistemul pentru a satisface o nevoie a utilizatorului. Ele pot veni sub diverse forme:

- **User Stories (în Agile):** „Ca utilizator, vreau să pot reseta parola pentru a-mi recupera accesul la cont.”
- **SRS (Software Requirement Specification):** Documente tehnice detaliate cu diagrame și reguli stricte.
- **Mockups/Wireframes:** Design-ul vizual al paginilor.

2. De ce analizăm cerințele exhaustiv?

Obiectivul nostru este să găsim **defecte de documentație**. O cerință proastă este „rădăcina” a 50% din bug-urile de producție. Căutăm:

- **Ambiguitatea:** Cuvinte precum „rapid”, „ușor”, „uneori”, „aproximativ”. (Ex: „Sistemul trebuie să se încarce rapid” – Ce înseamnă rapid? 1 secundă sau 10?).
- **Incompletenta:** Ce se întâmplă în cazul de eroare? (Ex: Cerința spune cum te loghezi, dar nu spune ce se întâmplă dacă introduci parola greșită de 5 ori).
- **Inconsistența:** Când două cerințe se bat în cap. (Ex: Pagina A spune că fontul e roșu, Pagina B spune că fontul e albastru).

3. Derivarea Condițiilor de Testare (Test Conditions)

Condiția de Testare este un element sau un eveniment care poate fi verificat. Este răspunsul la întrebarea: „**CE testăm?**”.

- **Cerință:** „Utilizatorul trebuie să se poată loga în OrangeHRM cu un username și o parolă valide.”
- **Condiții de Testare derivate:**
 - Verificarea logării cu date valide.

- Verificarea comportamentului la introducerea unui username greșit.
- Verificarea comportamentului la introducerea unei parole greșite.
- Verificarea sensibilității la majuscule (Case Sensitivity).
- Verificarea accesului cu câmpuri goale.

💡 Exemple Practice pentru Discuție (OrangeHRM)

Scenariul A: Cerința „Leneșă”

- **Cerință:** „Modulul de Leave (Concedii) trebuie să permită angajaților să ceară zile libere.”
- **Analiză QA:** Această cerință este un dezastru pentru un tester.
- **Întrebări de pus la curs:**
 - Ce tipuri de concediu există? (Medical, Odihnă, Fără plată?)
 - Cine aprobă cererea?
 - Se pot cere fracțiuni de zi (ore) sau doar zile întregi?
 - Ce se întâmplă dacă angajatul nu mai are zile disponibile?

Scenariul B: Trasabilitatea (Traceability)

2. **Concept:** Fiecare Condiție de Testare trebuie să fie legată de o Cerință.
3. **Discuție:** Dacă ai un test care nu verifică nicio cerință, înseamnă că testezi ceva inutil. Dacă ai o cerință care nu are niciun test asociat, înseamnă că acea funcționalitate este un risc major (netestată).

Știați că...?

💡 Există o tehnică numită „**Ambiguitatea Intenționată**”? Uneori, clienții lasă cerințele vagi pentru că nici ei nu știu exact cum vor să arate produsul final. Rolul tău ca QA este să „forțezi” claritatea. O întrebare pusă la timp de un tester poate salva mii de euro în ore de programare irosite pe o idee greșită.

Poveste Aplicată: „Rețeta Bucătarului și Clientul Pretențios”

Imaginează-ți că ești un critic culinar (QA) și primești o comandă de la un client pentru un bucătar (Developer).

- **Cerința Clientului:** „Vreau o supă bună și caldă.”

- **Analiza ta (QA):** Dacă îi dai această „cerință” bucătarului, el s-ar putea să facă o supă de pește la 40 de grade. Dar dacă clientul urăște peștele și voia supa la 80 de grade?
- **Acțiunea ta:** Te duci la client și analizezi:
 - Ce înseamnă „bună”? (Sărată, picantă, de pui?)
 - Ce înseamnă „caldă”? (Temperatura exactă).
 - Există alergii? (Condiții negative).

Abia după ce ai „disecat” cerința și ai scris: „Supă de pui, cu tăieței, servită la 75 de grade, fără pătrunjel”, bucătarul poate găti (Code), iar tu poți verifica (Test) dacă rezultatul este cel așteptat. Fără această analiză, bucătarul muncește degeaba, iar clientul rămâne nemulțumit.

Exercițiu de Gândire: „Detectivul de Cerințe”

Analizează următoarea User Story pentru **OrangeHRM** și identifică 3 ambiguități sau informații care lipsesc:

„Ca administrator, vreau să pot șterge utilizatori din sistem printr-un simplu click, pentru a curăța baza de date.”

1. **Ambiguitatea 1:** _____
2. **Ambiguitatea 2:** _____
3. **Ambiguitatea 3:** _____

(Sugestii pentru trainer: Ce se întâmplă cu datele istorice ale utilizatorului șters? Există un mesaj de confirmare „Sigur doriți să ștergeți?” sau se șterge instant? Poate un admin să se șteargă pe sine însuși?)

Capitolul 4: Test Case Design (Anatomia și Scrierea Cazurilor de Test)

 Dacă Planul de Testare este „Strategia de Război”, **Cazul de Test (Test Case)** este „Ordinul de Luptă” specific. Un caz de test este un set de condiții sau variabile sub care un tester va determina dacă un sistem software satisface cerințele sau funcționează corect. În

modelul QualiAdept, un Test Case bine scris trebuie să fie **atomic, independent și clar**.

1. Ce este un Caz de Test?

Un Test Case este un document care descrie un scenariu specific de utilizare a aplicației. Acesta transformă o „Condiție de Testare” (ex: Verificarea logării) într-o secvență de pași reproductibili.

Regula de aur: Un caz de test trebuie să poată fi executat de către oricine din echipă, fără a cere explicații suplimentare autorului.

2. Structura Exhaustivă a unui Caz de Test (Anatomia)

Fiecare câmp dintr-un instrument de Test Management (cum este Zephyr în Jira) are un scop precis. Să le analizăm în detaliu:

A. Test Case ID (Identificatorul Unic)

Este un cod alfanumeric care ajută la trasabilitate.

- *Format recomandat:* [Proiect]_[Modul]_[Număr]
- *Exemplu:* OHRM_LOGIN_001 (OrangeHRM, modulul Login, testul 1).

B. Titlu / Sumar (Title / Summary)

O descriere scurtă și clară a ceea ce verificăm. Trebuie să conțină „Ce”, „Unde” și, uneori, „Când”.

- *Greșit:* „Test de login”.
- *Corect (QualiAdept style):* „Verificarea logării cu succes folosind credențiale valide de Administrator”.

C. Precondiții (Preconditions)

Starea în care trebuie să se afle sistemul **înainte** de a începe pașii de testare.

- *Exemplu:* „Aplicația OrangeHRM este deschisă pe pagina de Login”, „Utilizatorul 'Admin' este deja înregistrat în baza de date”.

D. Pașii de Testare (Test Steps)

Instrucțiuni clare, acționabile. Se folosesc verbe la imperativ (Apasă, Introdu, Navighează).

- Introdu textul 'Admin' în câmpul 'Username'.

- Introdu textul 'admin123' în câmpul 'Password'.
- Apasă butonul 'Login'.

E. Date de Test (Test Data)

Valorile exacte folosite în timpul testului.

- *Exemplu:* Username: Admin, Password: admin123.

F. Rezultat Așteptat (Expected Result) - Cel mai important câmp!

Comportamentul pe care sistemul **ar trebui** să îl aibă conform cerințelor de business.

- *Exemplu:* „Utilizatorul este redirecționat către pagina Dashboard. Apare mesajul de întâmpinare 'Welcome Admin'”.

G. Rezultat Real (Actual Result)

Câmpul care se completează doar în momentul **execuției**. Dacă Rezultatul Real $\neq$ Rezultatul Așteptat, avem un Bug!

H. Post-condiții (Post-conditions)

Starea sistemului după finalizarea testului (curățenia de după).

4. *Exemplu:* „Utilizatorul se deloghează pentru a lăsa mediul curat pentru următorul test”.

3. Proprietățile unui Test Case de Calitate (Standarde QualiAdept)

Pentru a scrie teste profesionale, trebuie să respectăm aceste principii:

- **Atomicitate:** Un test trebuie să verifice un singur lucru. Nu combina „Logarea” cu „Schimbarea parolei” în același Test Case. Dacă eșuează, nu vei ști care parte e de vină.
- **Independență:** Testul nu ar trebui să depindă de rezultatul testului anterior.
- **Reproductibilitate:** Oricine îl rulează, pe orice mediu valid, trebuie să obțină același rezultat.
- **Trasabilitate:** Fiecare Test Case trebuie să fie legat de o cerință (User Story) din Jira.

Exemple Practice pentru Discuție (OrangeHRM)

Scenariul A: Testul „Leneș”

- **Pași:** 1. Loghează-te. 2. Verifică Dashboard-ul.
- **Analiză QA:** Acest test este prea vag. Ce username folosim? Ce înseamnă „verifică”? Ce ar trebui să vedem pe Dashboard? Un astfel de test va fi interpretat diferit de 3 testeri diferiți.

Scenariul B: Testul „Proză”

- **Pași:** „Te duci pe site, apoi te uiți unde scrie username și scrii acolo admin, apoi pui parola care ți-a fost dată și apeși pe butonul mare și portocaliu de login.”
- **Analiză QA:** Prea mult text inutil. Instrucțiunile trebuie să fie telegrafice și tehnice: „1. Introdu Username. 2. Introdu Parola. 3. Click Login.”

Știați că...?

💡 În industrie există conceptul de „**Step Overkill**”? Unii testeri scriu pași precum „1.

Deschide browserul. 2. Scrie adresa. 3. Apasă Enter. 4. Așteaptă să se încarce pagina.”

În realitate, acești pași pot fi incluși în **Precondiții**. Testul propriu-zis ar trebui să înceapă acolo unde începe funcționalitatea vizată. Economisirea timpului de citire crește productivitatea echipei cu până la 20%.

Poveste Aplicată: „Manualul de Asamblare IKEA”

Imaginează-ți că un Test Case este o pagină dintr-un manual de asamblare IKEA.

- **Precondiții:** Trebuie să ai toate piesele scoase din cutie și o șurubelniță (Resurse/Scule).
- **Pașii:** Sunt desene clare (Instrucțiuni) care îți spun exact ce șurub să pui în ce gaură.
- **Test Data:** Șurubul de 5mm (nu cel de 10mm).
- **Rezultatul Așteptat:** Două plăci de lemn sunt acum unite la un unghi de 90 de grade.

Dacă manualul IKEA ar fi scris prost (ex: „Pune scândurile împreună”), dulapul tău (Software-ul) ar ieși strâmb sau s-ar dărâma. Un tester profesionist scrie manuale de asamblare perfecte pentru ca software-ul să nu se dărâme la prima utilizare.



Exercițiu de Gândire: „Să scriem corect!”

Avem următoarea cerință: „Sistemul trebuie să permită resetarea parolei prin trimiterea unui cod pe email.”

Task: Definește **Precondițiile** și **Rezultatul Așteptat** pentru acest caz de test:

- **ID:** OHRM_AUTH_005
- **Titlu:** Verificarea primirii email-ului de resetare parolă.
- **Precondiții:** _____
- **Rezultat Așteptat:** _____

Capitolul 5: Tipuri de Teste și Piramida Testării

💡 Testarea software-ului nu este o activitate uniformă. Pentru a fi eficienți, trebuie să verificăm aplicația la diferite „altitudini”. În acest capitol, explorăm cele patru niveluri principale de testare, conceptul strategic al Piramidei Testării și rezolvăm una dintre cele mai mari dileme ale începătorilor: diferența dintre **Retesting** și **Regression**.

1. Nivelurile de Testare (Testing Levels)

Conform standardelor ISTQB, testarea se desfășoară în patru etape logice, pe măsură ce software-ul este construit:

A. Testarea Unitară (Unit Testing)

- **Ce testăm:** Cele mai mici componente izolate (funcții, metode, clase de cod).
- **Cine o face:** Dezvoltatorii (Developers).
- **Obiectiv:** Verificarea logicii interne a codului. Este cea mai ieftină etapă de reparare a bug-urilor.

B. Testarea de Integrare (Integration Testing)

- **Ce testăm:** Interacțiunea dintre două sau mai multe module care funcționează bine individual, dar pot avea probleme când „vorlesc” între ele.
- **Cine o face:** Dezvoltatori sau Testeri tehnici.
- **Obiectiv:** Identificarea defectelor în interfețele și fluxurile de date dintre componente.

C. Testarea de Sistem (System Testing)

- **Ce testăm:** Întregul sistem ca un tot unitar, bazat pe cerințele de business (SRS).
- **Cine o face:** Testerii QA (Manual sau Automat).
- **Obiectiv:** Verificarea conformității aplicației cu nevoile utilizatorului final. Aici intervine testarea End-to-End (E2E).

D. Testarea de Acceptanță (Acceptance Testing - UAT)

- **Ce testăm:** Validarea finală a sistemului.
- **Cine o face:** Clienții, Utilizatorii Finali sau Product Owner-ul.
- **Obiectiv:** Decizia de a „accepta” produsul pentru lansarea în producție. (Se răspunde la întrebarea: „Este acest produs util pentru afacerea mea?”).

2. Piramida Testării (The Testing Pyramid)

Conceptul de Piramidă a Testării (introdus de Mike Cohn) ne învață cum să distribuim efortul de testare pentru a obține cel mai bun Return on Investment (ROI).

- **Baza (Unit Tests):** Trebuie să avem mii de astfel de teste. Sunt rapide, automate și ne oferă feedback instant.
- **Mijlocul (API/Integration Tests):** Verifică logica de business fără a trece prin interfața grafică. Sunt mai lente decât cele unitare, dar mai rapide decât cele manuale.
- **Vârful (UI/Manual Testing):** Sunt testele cele mai scumpe și mai lente. Aici intră testarea manuală QualiAdept. Deși sunt puține ca număr, ele sunt vitale pentru a verifica experiența utilizatorului (UX).

Eroarea strategică: Multe echipe inversează piramida (se bazează doar pe teste manuale la

final), ceea ce duce la costuri uriașe și lansări întârziate.

3. Retesting vs. Regression Testing (Marea Distincție)

Aceasta este „piatra de încercare” la orice interviu de QA.

Caracteristică	Retesting (Confirmation Testing)	Regression Testing (Testarea de Regresie)
Definiție	Verificăm dacă un bug raportat anterior a fost fixat cu adevărat.	Verificăm dacă fixarea unui bug nu a stricat alte zone care funcționau.
Scop	Confirmarea reparației.	Protejarea integrității sistemului.
Când se face?	Imediat după ce Developerul spune „Fixed”.	După un Retesting reușit sau la final de ciclu.
Automatizare	Greu de automatizat (bug-uri punctuale).	Ideal pentru automatizare (repetitiv).

💡 Exemple Practice pentru Discuție (OrangeHRM)

Scenariul A: „Butonul de Save”

- **Context:** Ai raportat că butonul de „Save” din modulul Personal Details nu funcționează. Developerul îl repară.
- **Acțiunea 1 (Retesting):** Dai click pe „Save” să vezi dacă acum salvează. (A trecut!).
- **Acțiunea 2 (Regression):** Verifici dacă mai poți edita datele sau dacă poți să te mai deloghezi.
- **Întrebare de discuție:** De ce ar putea „Save”-ul să strice „Logout”-ul? (Răspuns: Conflicte de scripturi sau sesiuni corupte).

Scenariul B: Nivelurile de testare în OrangeHRM

5. **Unit:** Un dev verifică dacă funcția care calculează zilele de concediu rămase returnează 21

- 5 = 16.

6. **Integration:** Verificăm dacă modulul „Leave” trimite corect notificarea către modulul „Dashboard”.
7. **System:** Testerul QA face un flux complet: Angajare -> Cerere Concediu -> Aprobare -> Verificare Stat de plată.

Știați că...?

💡 Există un tip de testare numit „**Sanity Testing**”? Este o formă scurtă de regresie care se face când nu avem timp să testăm tot. Dacă regresia completă durează 2 zile, un „Sanity” durează 2 ore și verifică doar fluxurile vitale (ex: „Dacă nu ne putem loga, nu mai testăm restul”).

Poveste Aplicată: „Construirea unui Smartphone”

Imaginează-ți că ești șeful calității la o fabrică de telefoane mobile.

- **Unit Testing:** Inginerii verifică fiecare piesă individual: ecranul se aprinde? Difuzorul scoate sunet? Bateria se încarcă pe bancul de probe?
- **Integration Testing:** Conectăm ecranul la placa de bază. Mai funcționează amândouă împreună? Sau placa de bază „arde” ecranul?
- **System Testing:** Asamblăm tot telefonul. Punem carcasa, pornim Android-ul și verificăm: Pot să dau un apel? Merge Wi-Fi-ul? Camera face poze clare?
- **Acceptance Testing (UAT):** Trimitem telefonul la un grup de utilizatori obișnuiți. Ei spun: „E frumos, dar e prea greu” sau „Butonul de volum e prea sus”. Pe baza feedback-ului lor, decidem dacă intrăm în producție de serie.

Morala QualiAdept: Dacă ai sărit peste Unit Testing (nu ai verificat bateria) și ai descoperit că bateria explodează abia la System Testing (când telefonul e gata), ai pierdut mii de euro pe carcase și ecrane distruse de o piesă defectă. **Testarea pe niveluri înseamnă economie și siguranță!**

Exercițiu de Gândire: „Detectivul de Regresie”

S-a făcut o actualizare la logo-ul companiei în header-ul paginii OrangeHRM (o modificare pur estetică).

- Ce ai face ca **Retesting**?

- Ce ai verifica la **Regresie**? (Gândește-te: header-ul apare pe toate paginile? Ar putea să acopere butoanele de meniu pe rezoluții mici?)

Capitol Bonus: Testarea Exploratorie vs. Testarea Scriptată (Intuiție vs. Structură)

🧠 În capitolele anterioare, am învățat cum să scriem cazuri de test detaliate și cum să planificăm totul „la virgulă”. Totuși, în realitate, cele mai spectaculoase bug-uri sunt găsite adesea în afara acestor documente. În acest capitol bonus, explorăm dualitatea dintre **Testarea Scriptată** (procesul formal) și **Testarea Exploratorie** (arta descoperirii).

1. Testarea Scriptată (Scripted Testing)

Este ceea ce am exersat până acum: scrierea de Test Case-uri bazate pe cerințe, cu pași preciși și rezultate așteptate predefinite.

- **Avantaje:**
 - Oferă o dovadă clară a acoperirii cerințelor (Traceability).
 - Este ușor de delegat și de raportat (știm exact câte teste au trecut/eșuat).

- Este baza pentru automatizarea viitoare.
- **Dezavantaje:**
 - **Efectul de „tunel”:** Testerul urmărește doar pașii scriși și poate ignora un bug evident care se află la un centimetru de click-ul său.
 - Consumă mult timp pentru documentare.

2. Testarea Exploratorie (Exploratory Testing)

Definiție: Un stil de testare unde învățarea, designul testelor și execuția au loc simultan. Testerul nu are un set de pași scriși, ci o **Cartă (Charter)** – un obiectiv general.

- **Mecanism:** Testerul explorează aplicația bazându-se pe experiență, intuiție și pe ceea ce descoperă „pe parcurs”.
- **Avantaje:**
 - Găsește bug-uri complexe de tip „edge case” pe care nicio cerință nu le-a prevăzut.
 - Este extrem de rapidă (nu necesită documentație prealabilă).
 - Stimulează creativitatea și mindset-ul de „detectiv”.

Comparație: Când folosim ce?

Situație	Testare Scriptată	Testare Exploratorie
Obiectiv	Verificarea conformității (Merge conform cerinței?)	Descoperirea necunoscutului (Unde se poate rupe?)
Documentație	Test Case-uri detaliate	Notițe scurte, screenshot-uri, log-uri
Moment	Regresie, faze formale de System Testing	După ce testele scriptate au trecut, sau în faze critice
Skill necesar	Atenție la detalii, rigoare	Intuiție, experiență bogată, gândire laterală

Exemple Practice pentru Discuție (OrangeHRM)

Scenariul A: Scriptat (Modulul Personal Details)

- **Test Case:** „Introdu numele 'Popescu', apasă Save, verifică dacă s-a salvat.”
- **Rezultat:** Testul trece. Totul pare OK.

Scenariul B: Exploratoriu (Modulul Personal Details)

8. **Abordare:** Testerul se gândește: „Ce se întâmplă dacă încerc să salvez numele în timp ce dau click rapid pe butonul de Edit al altui câmp? Sau dacă pun un emoji în nume?”.
9. **Rezultat:** Descoperă că baza de date nu suportă caractere speciale și aplicația afișează un cod de eroare urât (Internal Server Error).
10. **Concluzie QualiAdept:** Testul scriptat a confirmat că funcționează. Testul exploratoriu a arătat cum se strică.

Știați că...?

💡 Există o metodă numită **Session-Based Test Management (SBTM)**? Aceasta este modul profesional de a face testare exploratorie: setezi un cronometru (ex: 90 de minute), alegi un obiectiv (ex: „Voi explora doar fluxul de resetare parolă”) și la final raportezi ce ai învățat și ce bug-uri ai găsit. Nu este „joacă”, ci explorare disciplinată.

Poveste Aplicată: „Turistul vs. Exploratorul”

Imaginează-ți că vizitezi un oraș nou (Aplicația Software).

- **Turistul (Testarea Scriptată):** Ai un ghid turistic (Test Plan) și o listă de obiective (Test Case-uri). Mergi la Turnul Eiffel, apoi la Luvru, apoi la Arcul de Triumf. Vezi exact ce scrie în carte. La final, poți spune: „Am vizitat Parisul conform planului”.
- **Exploratorul (Testarea Exploratorie):** Ai văzut obiectivele principale, dar apoi decizi să o iei pe o străduță lăaturalnică pentru că ai văzut o pisică interesantă. Găsești o brutărie ascunsă unde se face cea mai bună baghetă din lume, pe care niciun ghid nu o menționează.

Morala: Ca QA, trebuie să fii ambele. Turistul se asigură că am văzut „vedetele” aplicației (cerințele), iar Exploratorul găsește micile (sau marile) probleme ascunse în locuri în care nimeni

nu s-a gândit să se uite.

Exercițiu de Gândire: „Dincolo de pași”

Avem un caz de test scriptat pentru **OrangeHRM**: „Introducerea unei poze de profil”.

Pașii spun: 1. Click 'Browse', 2. Selectează 'photo.jpg', 3. Click 'Upload'.

Dacă ai avea 10 minute de **Testare Exploratorie** pe această funcționalitate, ce „străduțe lăturalnice” ai explora?

- *Exemple de gândire:* „Dacă pun un fișier de 50GB?”, „Dacă pun un fișier .exe redenumit în .jpg?”, „Dacă apăs Upload de 10 ori consecutiv?”.

Capitolul 6: Recapitulare, Concluzii și Proiectul OrangeHRM

 Felicitări! Ai parcurs fundamentele planificării și designului în testare. Dacă Sesiunea 1 te-a învățat „de ce” testăm, Sesiunea 2 ți-a oferit răspunsul la întrebarea „cum” ne organizăm pentru a fi eficienți. Un tester fără plan este ca un călător fără busolă; poate va ajunge la destinație, dar cu siguranță va irosi mult timp și energie pe drum.

1. Recapitulare Exhaustivă: Ce am învățat în Sesiunea 2?

- **Planul de Testare:** Harta strategică a proiectului. Am învățat să definim **Scope** (ce testăm și ce nu), să stabilim **Criteriile de Intrare/ieșire** și să ne asumăm roluri clare în echipă.
- **Analiza Cerințelor:** Am devenit detectivi de documentație. Știm să „vănăm” ambiguitățile și să transformăm o cerință vagă într-o **Condiție de Testare** clară.
- **Test Case Design:** Am învățat anatomia unui caz de test (ID, Precondiții, Pași, Rezultat Așteptat). Știm că un test bun trebuie să fie **atomic** și **reproductibil**.
- **Nivelurile și Piramida Testării:** Am înțeles că baza calității stă în testele unitare (făcute de developeri), iar noi, QA-ul, verificăm sistemul în ansamblu.
- **Retesting vs. Regresie:** Am clarificat marea dilemă. Retesting-ul confirmă reparația, Regresia confirmă că „nu s-a stricat altceva”.

- **Testare Scriptată vs. Exploratorie:** Am învățat să combinăm rigoarea pașilor scriși cu intuiția descoperirii libere.

2. Concluziile QualiAdept: „Legile Strategului QA”

- **Scope-ul este protecția ta.** Dacă nu este scris în Plan că testezi pe iPhone 8, nimeni nu te poate învinovăți că nu ai găsit un bug acolo.
- **Cerințele ambigue nasc bug-uri certe.** Clarifică totul înainte ca programatorul să scrie prima linie de cod.
- **Un Test Case fără Rezultat Așteptat este doar o sugestie.** Rezultatul Așteptat este singurul care definește succesul sau eșecul.
- **Regresia nu este opțională.** Software-ul este un organism interconectat; o modificare la „Login” poate dăruia „Plățile” fără niciun avertisment.

3. Temă de Casă: „Proiectul OrangeHRM - Faza de Design”

Această temă reprezintă începutul proiectului tău de portofoliu. Vom folosi aplicația **OrangeHRM** (varianta Demo sau instalată local).

Task 1: Fragment de Plan de Testare (Modulul PIM - Personal Information Management)

Definește următoarele elemente pentru testarea modulului PIM:

- **In-Scope:** Menționează 3 funcționalități care trebuie testate.
- **Out-of-Scope:** Menționează 2 elemente pe care decizi să NU le testezi (ex: o anumită rezoluție, un sub-modul nefinalizat).
- **Exit Criteria:** Scrie 2 condiții care trebuie îndeplinite pentru a putea spune: „Am terminat testarea modulului PIM”.

Task 2: Analiza de Cerințe (Cazul „Adăugare Angajat”)

Cerința primită de la client: *„Sistemul trebuie să permită adăugarea unui angajat nou prin completarea numelui și a unui ID unic.”*

- Identifică **2 ambiguități** în această cerință.
- Pune **2 întrebări** de clarificare pe care i le-ai adresa Product Owner-ului.

Task 3: Design de Cazuri de Test (Execuție)

Scrie **3 Cazuri de Test** complete pentru funcționalitatea de **Login** în OrangeHRM, folosind

structura: ID, Titlu, Precondiții, Pași, Date de Test, Rezultat Așteptat.

- **Test 1:** Happy Path (Login cu succes).
- **Test 2:** Negative Path (Parolă greșită).
- **Test 3:** Edge Case/Validation (Câmpuri goale).

Task 4: Strategie de Regresie

Imaginează-ți că s-a modificat doar culoarea butonului de „Login”.

- Ce faci pentru **Retesting**?
- Alege **un singur alt modul** din aplicație pe care l-ai verifica la **Regresie** și justifică de ce crezi că ar putea fi afectat.

Sfat de final de la echipa QualiAdept

Nu încerca să scrii „cel mai lung” Plan de Testare. Scrie-l pe cel mai util. În automatizarea și testarea modernă, simplitatea și claritatea bat întotdeauna cantitatea.

Ne vedem la **Sesiunea 3**, unde vom învăța cum să raportăm bug-urile pe care le vom găsi folosind aceste planuri!